Salesforce Developer

Complete Curriculum Notes

Theory + Practice + Examples
From Apex Fundamentals to Lightning Web Components

Covering Days 24 to 49 of the Curriculum
Apex • SOQL/SOSL • Triggers • Async • Integration • LWC

Table of Contents

1. Introduction to Apex
2. Data Types and Logic Control in Apex
3. Apex Classes
4. sObject Relationships
5. SOQL & SOSL
6. Data Manipulation Language (DML)
7. Apex Debugging
8. Apex Triggers, Events, Context Variables, Trigger Framework
9. Trigger Order of Execution
10. Exception Handling
11. Test Classes
12. Synchronous vs Asynchronous Operations
13. Apex Annotations & Interfaces (@future, Batchable, Schedulable)
14. Queueable & @InvocableMethod
15. Deployment using Change Sets
16. Integration — SOAP, REST, Patterns, Postman, LLM
17. ECMA 6+ — JavaScript for Salesforce Devs
18. Promises, Async/Await, Modules, JSON
19. VS Code & SFDX Setup
20. Introduction to LWC — Framework, Aura vs LWC, Bundle
21. Data Binding, Decorators, CSS, Rendering, Loops
22. JS Properties, Component Lifecycle & Composition
23. Events — Component Communication (Parent-Child, Child-Parent, LMS)
24. Accessing Salesforce Data (LDS, JS API, Wire, Apex), Navigation Service

1. Introduction to Apex

What is Apex?

Apex is Salesforce's **proprietary, strongly-typed, object-oriented programming language** that runs on the Salesforce servers (multi-tenant cloud). Its syntax looks very similar to Java, but it is purpose-built to interact with the Salesforce database and platform features.

> ***Analogy:*** *If Salesforce is a grand hotel, then declarative tools (Flow, Process Builder, Validation Rules) are the hotel staff who can handle most guest requests. But when a request is too complex — say, a guest wants a custom 7-course meal cooked exactly to their grandmother's recipe — you need a chef.* ***Apex is that chef.***

Key Characteristics

- **Hosted** — runs entirely on Salesforce servers, you never install it on your laptop.
- **Multi-tenant aware** — Salesforce enforces strict *governor limits* so your code can't hog shared resources.
- **Database-integrated** — SOQL queries and DML statements are part of the language itself, not an external library.
- **Object-oriented** — supports classes, interfaces, inheritance, polymorphism.
- **Strongly typed** — every variable has a declared type, checked at compile time.
- **Case-insensitive** — `Account` and `account` are treated the same.
- **Automatically tested** — Salesforce requires at least 75% code coverage before any Apex can be deployed to production.

When to Use Apex

Use Apex when...	Use Declarative when...
Complex business logic that loops, branches, recurses	Simple field updates or record creations
Calling external APIs	Sending email alerts
Running batch jobs over millions of records	Validation rules with formulas
Custom Lightning Component back-end	Approval processes

> **Best Practice:** Always try declarative first. Use code only when point-and-click cannot do the job. This is Salesforce's "*clicks before code*" philosophy.

Hello World in Apex

```
// Open Developer Console → Debug → Open Execute Anonymous Window
String greeting = 'Hello, Salesforce!';
System.debug(greeting);

// Output appears in the Debug Logs:
// USER_DEBUG|[2]|DEBUG|Hello, Salesforce!
```

Where Apex Lives

- **Apex Classes** — reusable units of code (most common).
- **Apex Triggers** — code that automatically fires before/after DML on a specific sObject.
- **Anonymous Blocks** — one-off scripts run in Developer Console.
- **Test Classes** — code whose only purpose is to test other code.

> **Governor Limits — the price of multi-tenancy:** Because thousands of customers share the same Salesforce hardware, every transaction is capped. For example, in a single transaction you can run a maximum of **100 SOQL queries**, **150 DML statements**, and process up to **50,000 records**. Hitting these limits throws an unrecoverable exception. Writing efficient, bulkified code is therefore not optional — it's survival.

2. Data Types and Logic Control in Apex

Primitive Data Types

Type	Description	Example
`Integer`	32-bit whole number	`Integer age = 25;`
`Long`	64-bit whole number	`Long population = 7000000000L;`
`Double`	64-bit floating-point	`Double pi = 3.14159;`
`Decimal`	Arbitrary-precision number (great for currency)	`Decimal price = 99.99;`
`Boolean`	true / false / null	`Boolean isActive = true;`
`String`	Sequence of characters (max 6,000,000 chars)	`String name = 'Salesforce';`
`Date`	Calendar date (no time)	`Date today = Date.today();`
`Datetime`	Date + time	`Datetime now = Datetime.now();`
`Time`	Time of day	`Time t = Time.newInstance(12,30,0,0);`
`Id`	15- or 18-character Salesforce record ID	`Id accId = '001xx0000000abc';`
`Blob`	Binary data (files, attachments)	`Blob b = Blob.valueOf('hi');`

Collections

Apex has three powerful collection types. Mastering these is the single highest-leverage thing you can do early on.

List

Ordered, allows duplicates, indexed from 0. Equivalent to a Java `ArrayList`.

```
List<String> fruits = new List<String>();
fruits.add('Apple');
fruits.add('Banana');
fruits.add('Apple');           // duplicates OK
System.debug(fruits[0]);       // Apple
System.debug(fruits.size());   // 3

// Inline initialization
List<Integer> nums = new List<Integer>{10, 20, 30};
```

Set

Unordered, **no duplicates**, no index. Perfect for "do I already have this?" checks.

```
Set<String> emails = new Set<String>();
emails.add('a@x.com');
emails.add('a@x.com');         // silently ignored
System.debug(emails.size());   // 1
System.debug(emails.contains('a@x.com')); // true
```

Map

Key-value pairs. Indispensable for relating records to each other quickly.

```
Map<Id, Account> accountById = new Map<Id, Account>();
for (Account a : [SELECT Id, Name FROM Account LIMIT 10]) {
    accountById.put(a.Id, a);
}
Account first = accountById.get(someId);
```

> **Pro Pattern:** `Map<Id, sObject>` is the most-used pattern in production Apex. It lets you avoid nested loops (which kill performance).

Operators

- **Arithmetic:** `+ - * / Math.mod()`
- **Comparison:** `== != < > <= >=` (Apex `==` is value equality, not reference — different from Java!)
- **Logical:** `&& || !` (short-circuit evaluation)
- **Assignment:** `= += -= *= /=`
- **Ternary:** `condition ? valueIfTrue : valueIfFalse`

Control Flow

if / else if / else

```
Decimal grade = 78;
if (grade >= 90)        System.debug('A');
else if (grade >= 80)   System.debug('B');
else if (grade >= 70)   System.debug('C');
else                    System.debug('F');
```

switch (Apex flavor)

```
String role = 'Admin';
switch on role {
    when 'Admin'              { System.debug('full access'); }
    when 'Manager','TeamLead' { System.debug('elevated access'); }
    when else                 { System.debug('basic access'); }
}
```

Loops

```
// Traditional for
for (Integer i = 0; i < 5; i++) System.debug(i);

// Enhanced for (most common)
for (Account a : [SELECT Id, Name FROM Account LIMIT 100]) {
    System.debug(a.Name);
}

// while
Integer count = 0;
while (count < 3) { System.debug(count); count++; }

// do-while runs at least once
Integer x = 10;
do { System.debug(x); x++; } while (x < 5);   // prints 10 once
```

> **Beware:** Never put SOQL queries or DML statements *inside* a loop. This is the #1 cause of governor-limit failures in production. Always query first, then loop.

Practical Example — Apply a Discount

```
List<Opportunity> opps = [SELECT Id, Amount, StageName
                          FROM Opportunity
                          WHERE StageName = 'Negotiation' LIMIT 50];

for (Opportunity o : opps) {
    if (o.Amount != null && o.Amount > 100000) {
        o.Amount = o.Amount * 0.95;   // 5% strategic discount
    }
}
update opps;   // ONE DML, outside the loop ✅
```

3. Apex Classes

An Apex **class** is a blueprint for creating objects (instances). It bundles together *data* (variables) and *behavior* (methods).

> ***Analogy:*** *A class is like a cookie cutter; objects are the cookies. The cutter defines shape; each cookie has its own dough (data).*

Anatomy of a Class

```
public with sharing class Car {

    // 1. Instance variables (each Car object has its own)
    public String make;
    public String model;
    private Integer speed;

    // 2. Static variable (shared across ALL Car objects)
    public static Integer totalCarsCreated = 0;

    // 3. Constructor — runs when "new Car(...)" is called
    public Car(String make, String model) {
        this.make = make;
        this.model = model;
        this.speed = 0;
        totalCarsCreated++;
    }

    // 4. Instance method
    public void accelerate(Integer kmph) {
        this.speed += kmph;
    }

    // 5. Static method (called on the class, not the object)
    public static Integer getCarCount() {
        return totalCarsCreated;
    }
}
```

Using It

```
Car c1 = new Car('Toyota', 'Camry');
Car c2 = new Car('Tesla',  'Model 3');
c1.accelerate(60);
System.debug(c1.speed);              // 60
System.debug(Car.getCarCount());     // 2
```

Access Modifiers

Modifier	Visible from
`private`	Same class only (default for variables/methods)
`protected`	Same class + inner/sub-classes
`public`	Anywhere in the same Salesforce org
`global`	Anywhere — even other orgs / managed packages

Sharing Keywords (Apex-specific!)

- `with sharing` — code respects the running user's record-level security.
- `without sharing` — code ignores sharing rules (use carefully!).
- `inherited sharing` — adopts caller's setting (safest default for utility classes).

> **Security:** Always use `with sharing` unless you have a deliberate reason not to. Forgetting this is a common Salesforce security audit finding.

Inheritance

Apex supports single inheritance via `extends` and multiple-interface implementation via `implements`. Use `virtual` on a parent to allow override; `override` on the child.

```
public virtual class Animal {
    public virtual void speak() { System.debug('Some sound'); }
}
public class Dog extends Animal {
    public override void speak() { System.debug('Woof!'); }
}
Animal a = new Dog();
a.speak();   // Woof!  — polymorphism in action
```

Interfaces

An interface is a contract — a list of method signatures with no body. Any class that `implements` it must provide bodies.

```
public interface Payable {
    Decimal calculatePay();
}
public class Employee implements Payable {
    public Decimal hourlyRate;
    public Decimal hoursWorked;
    public Decimal calculatePay() { return hourlyRate * hoursWorked; }
}
```

Inner Classes

A class defined inside another class. Often used to create lightweight "data carriers" (called wrappers).

```
public class AccountWrapper {
    public Account record;
    public Boolean isSelected;
}
```

> **Wrapper-class trick:** When you need to display Salesforce records in a Lightning Component *plus* a checkbox or extra UI flag, return a list of these wrappers from your Apex method.

4. sObject Relationships

An **sObject** is any Salesforce database object — Account, Contact, Case, custom `Invoice__c`, etc. Records related to each other form Salesforce's data backbone.

Two Main Relationship Types

1. Lookup Relationship

- Loose connection between two objects.
- Child can exist *without* a parent.
- Up to 40 lookups per object.

> ***Analogy:*** *Loosely linked like Facebook friendships — you can unfriend, both still exist.*

2. Master-Detail Relationship

- Tight, parent-driven connection.
- Child **cannot exist without** a parent — delete parent → all children deleted (cascading delete).
- Child inherits parent's sharing & security.
- Enables **roll-up summary fields** (SUM, COUNT, MIN, MAX on parent).
- Max 2 master-detail relationships per object.

> ***Analogy:*** *Like a folder and the files inside it — delete the folder, files vanish too.*

Special: Hierarchical (User only) & Many-to-Many

For many-to-many you create a **junction object** with two master-detail fields. Classic example: `Course` ↔ `CourseEnrollment__c` ↔ `Student`.

Querying Relationships in SOQL

Child → Parent (dot notation)

```
SELECT Id, Name, Account.Name, Account.Industry
FROM Contact
WHERE Account.Industry = 'Technology'
```

Parent → Children (subquery)

```
SELECT Name,
       (SELECT Id, FirstName, LastName FROM Contacts)
FROM Account
```

Note that the relationship name in the inner query is plural and uses the *relationship name*, not the object name. For custom objects it ends in `__r` (e.g., `Invoices__r`).

Up to 5 Levels

```
SELECT Contact.Account.Owner.Manager.Name FROM Case
```

> **Custom-object naming:** Field API names end in `__c`; relationship names end in `__r`. So `Invoice__c.Customer__c` stores the lookup ID, and `Invoice__c.Customer__r.Name` traverses it.

5. SOQL & SOSL

SOQL — Salesforce Object Query Language

Used to query **structured data** from a **single sObject** (or its related objects).

Syntax Basics

```
SELECT Id, Name, Industry, AnnualRevenue
FROM Account
WHERE Industry = 'Technology'
   AND AnnualRevenue > 1000000
ORDER BY AnnualRevenue DESC
LIMIT 100
OFFSET 10
```

SOQL in Apex

```
List<Account> accs = [SELECT Id, Name FROM Account WHERE Industry = 'Tech'];
Integer cnt = [SELECT COUNT() FROM Contact];
Account one = [SELECT Id FROM Account LIMIT 1];   // auto-cast
```

Dynamic SOQL

Build query strings at run time. Sanitize user input!

```
String safe = String.escapeSingleQuotes(userInput);
String q = 'SELECT Id, Name FROM Account WHERE Name LIKE \'%' + safe + '%\'';
List<Account> results = Database.query(q);
```

Bind Variables

```
String industry = 'Tech';
List<Account> accs = [SELECT Id FROM Account WHERE Industry = :industry];

Set<Id> ids = new Set<Id>{...};
List<Contact> cons = [SELECT Id FROM Contact WHERE AccountId IN :ids];
```

Aggregate Queries

```
List<AggregateResult> ar = [
    SELECT Industry, COUNT(Id) total, AVG(AnnualRevenue) avgRev
    FROM Account
    GROUP BY Industry
    HAVING COUNT(Id) > 5
];
for (AggregateResult r : ar) {
    System.debug(r.get('Industry') + ' → ' + r.get('total'));
}
```

SOSL — Salesforce Object Search Language

Used to search **across multiple sObjects** using a free-text search index. Best for "find anything matching keyword" use cases — like a global search bar.

```
List<List<sObject>> results = [
    FIND 'Acme' IN ALL FIELDS
    RETURNING Account(Name), Contact(FirstName, LastName), Lead(Company)
];

List<Account> accs    = (List<Account>)results[0];
List<Contact> cons    = (List<Contact>)results[1];
List<Lead>    leads   = (List<Lead>)results[2];
```

SOQL vs SOSL

Aspect	SOQL	SOSL
Searches	One object (and related)	Multiple objects
Data type	Specific fields	Indexed text
Returns	`List<sObject>`	`List<List<sObject>>`
Best for	Filtering/reporting	Free-text search

6. Data Manipulation Language (DML)

DML = how Apex modifies records: `insert`, `update`, `upsert`, `delete`, `undelete`, `merge`.

Insert

```
Account a = new Account(Name = 'Acme Corp', Industry = 'Technology');
insert a;
System.debug(a.Id);    // auto-populated after insert
```

Update

```
Account a = [SELECT Id, Name FROM Account LIMIT 1];
a.Name = 'New Name';
update a;
```

Upsert (insert OR update)

```
// If a.Id is null → insert; else → update
upsert a;

// Or use an external ID field as the key
upsert accountList External_Id__c;
```

Delete & Undelete

```
delete oldAccount;                    // goes to recycle bin
undelete [SELECT Id FROM Account WHERE Name='Restored' ALL ROWS];
```

Two Flavors of DML

1. Direct DML

Throws an exception on any failure — entire transaction rolls back.

```
insert accountList;
```

2. Database Methods

Allow **partial success** via the `allOrNone` flag.

```
Database.SaveResult[] results = Database.insert(accList, false);
for (Database.SaveResult r : results) {
    if (!r.isSuccess()) {
        for (Database.Error e : r.getErrors()) System.debug(e.getMessage());
    }
}
```

Bulkification — The Golden Rule

> **Never DML inside a loop.** The Apex governor allows only 150 DML statements per transaction.

```
// ✗ BAD — one DML per iteration
for (Account a : accList) { update a; }

// ✓ GOOD — single DML on the whole list
update accList;
```

7. Apex Debugging

Debugging in Apex relies primarily on the **Debug Log**, plus tools like the Developer Console, VS Code's Apex Replay Debugger, and Checkpoints.

System.debug()

```
System.debug('Account name: ' + acc.Name);
System.debug(LoggingLevel.ERROR, 'Critical error: ' + e.getMessage());
```

Debug Log Levels (lowest → highest verbosity)

- NONE → ERROR → WARN → INFO → DEBUG → FINE → FINER → FINEST

Reading the Log

Setup → Debug Logs → New. Set a user, time, log levels. Run your code, refresh, click the log to inspect:

- **EXECUTION_STARTED** / **FINISHED** — transaction boundaries.
- **USER_DEBUG** — your `System.debug` output.
- **SOQL_EXECUTE_BEGIN/END** — every query and its row count.
- **LIMIT_USAGE_FOR_NS** — governor-limit consumption summary at the end.

Limits Class — Live Limit Inspection

```
System.debug('Queries used: ' + Limits.getQueries() + '/' + Limits.getLimitQueries());
System.debug('DML rows: '     + Limits.getDmlRows() + '/' + Limits.getLimitDmlRows());
System.debug('CPU time ms: '  + Limits.getCpuTime() + '/' + Limits.getLimitCpuTime());
```

Checkpoints (Developer Console)

Click the gutter next to a line; when execution reaches it the entire heap state is snapshotted. Useful for reproducing tricky null pointers without touching the code.

Apex Replay Debugger (VS Code)

Generate a debug log, open it in VS Code → *SFDX: Launch Apex Replay Debugger with Current File*. Step line by line through the past execution, inspect variables.

> **Pattern:** Wrap risky logic in `try/catch`, log `e.getMessage()`, `e.getStackTraceString()`, and `e.getLineNumber()` — these three lines turn ghost bugs into 30-second fixes.

8. Apex Triggers, Events, Context Variables & Trigger Framework

A **trigger** is Apex code that automatically fires when records are inserted, updated, deleted, or undeleted on a particular sObject. It's how you enforce business rules that go beyond declarative tools.

Trigger Anatomy

```
trigger AccountTrigger on Account (before insert, before update,
                                   after insert, after update,
                                   before delete, after delete,
                                   after undelete) {

    if (Trigger.isBefore && Trigger.isInsert) {
        for (Account a : Trigger.new) {
            if (a.Industry == null) a.Industry = 'Other';
        }
    }
}
```

Seven Trigger Events

Event	Use Case
`before insert`	Default values, validations before save
`before update`	Modify fields, prevent invalid changes
`before delete`	Block deletion based on conditions
`after insert`	Need new record's Id (e.g., create child records)
`after update`	Roll-ups, cross-object updates
`after delete`	Cleanup related records, audit logs
`after undelete`	Restore related data

Before vs After:

- *Before* — record not yet saved, you can modify `Trigger.new` directly, NO DML needed.
- *After* — record saved, has Id, you must use DML to update `Trigger.new`. Records are read-only here.

Trigger Context Variables

Variable	Description
`Trigger.new`	List of new versions (insert/update/undelete)
`Trigger.old`	List of old versions (update/delete)
`Trigger.newMap`	Map<Id, sObject> of new (not in before-insert)
`Trigger.oldMap`	Map<Id, sObject> of old
`Trigger.isInsert / isUpdate / isDelete / isUndelete`	Operation flags
`Trigger.isBefore / isAfter`	Timing flags
`Trigger.isExecuting`	True when running inside a trigger
`Trigger.size`	Total records being processed

Recursive Triggers

Recursion happens when a trigger updates records, which fires the same trigger again, which updates more records... Apex stops at 16 levels with an error.

Static Variable Pattern

```
public class TriggerHelper {
    public static Boolean alreadyRun = false;
}

trigger AccountTrigger on Account (after update) {
    if (TriggerHelper.alreadyRun) return;
    TriggerHelper.alreadyRun = true;
    // ... your logic ...
}
```

Trigger Framework — One Trigger Per Object

Industry best practice: write a **handler class** that does the actual work, and keep the trigger itself paper-thin. Reasons: testability, readability, ordering of operations.

Handler Pattern Example

```
trigger AccountTrigger on Account (before insert, after insert,
                                   before update, after update) {
    AccountTriggerHandler.run();
}

public class AccountTriggerHandler {
    public static void run() {
        if (Trigger.isBefore && Trigger.isInsert)  setDefaults(Trigger.new);
        if (Trigger.isAfter  && Trigger.isInsert)  createWelcomeTask(Trigger.new);
        if (Trigger.isBefore && Trigger.isUpdate)  validateChanges(Trigger.new, Trigger.oldMap);
    }
    public static void setDefaults(List<Account> accs) {
        for (Account a : accs) if (a.Industry == null) a.Industry = 'Other';
    }
    // ... other methods ...
}
```

Practical Example — Welcome Task on New Account

```
trigger AccountTrigger on Account (after insert) {
    List<Task> tasks = new List<Task>();
    for (Account a : Trigger.new) {
        tasks.add(new Task(
            Subject  = 'Welcome call: ' + a.Name,
            WhatId   = a.Id,
            Priority = 'High',
            Status   = 'Not Started',
            ActivityDate = Date.today().addDays(3)
        ));
    }
    if (!tasks.isEmpty()) insert tasks;
}
```

Best Practices: 1 trigger/object, no logic in trigger body, always bulkify, no SOQL/DML inside loops, prevent recursion, use a framework (e.g. fflib), follow the order of execution rules.

9. Trigger Order of Execution

When you save a record, Salesforce executes a precise, well-known sequence of steps. Understanding this order saves countless hours of debugging.

The Full Sequence

1. Server loads original record (for updates).
2. System Validation (required fields, max length, valid IDs).
3. **Before Triggers** execute.
4. Custom validation rules run.
5. Duplicate rules run.
6. Record saved (NOT committed yet) — it now has an Id.
7. **After Triggers** execute.
8. Assignment Rules (Lead/Case).
9. Auto-Response Rules.
10. Workflow Rules — if any field updates, before/after triggers fire *once more*.
11. Process Builder & Flow (record-triggered) execute.
12. Escalation Rules (Cases).
13. Roll-Up Summary fields recalculated → may trigger parent's triggers/workflows.
14. Criteria-Based Sharing recalculated.
15. Database commit — data now permanent.
16. Post-commit logic (email alerts, async Apex, outbound messages).

> **Common gotcha:** Workflow field updates re-run before/after triggers. If you have a counter incrementing in a trigger you can double-count without noticing.

Practical Implication

If logic A must run *before* logic B, decide based on the order: a Process Builder change happens after triggers, so a trigger using that field will see the OLD value.

10. Exception Handling

Exceptions are runtime errors. If untrapped, they roll back the entire transaction and bubble a "`Apex script unhandled exception`" email to the developer.

Basic Try / Catch / Finally

```
try {
    Account a = [SELECT Id FROM Account WHERE Name='NoSuch' LIMIT 1];
} catch (QueryException e) {
    System.debug('Not found: ' + e.getMessage());
} catch (Exception e) {
    System.debug('Other error: ' + e.getMessage());
} finally {
    System.debug('Always runs (cleanup, log).');
}
```

Common Built-in Exception Types

Exception	When it fires
`NullPointerException`	Calling method/field on null variable
`QueryException`	Query returned 0 or >1 rows when assigning to single sObject
`DmlException`	Insert/update/delete failure (validation, duplicate, FLS)
`ListException`	Index out of bounds
`LimitException`	Governor limit hit (NOT catchable in most cases!)
`SObjectException`	Field accessed but not queried
`StringException`	Bad String operation
`TypeException`	Cast failure

Custom Exceptions

```
public class InsufficientFundsException extends Exception {}

public void withdraw(Decimal amount) {
    if (amount > balance) {
        throw new InsufficientFundsException('Balance too low: ' + balance);
    }
}
```

Useful Exception Methods

```
catch (Exception e) {
    System.debug(e.getMessage());
    System.debug(e.getStackTraceString());
    System.debug(e.getLineNumber());
    System.debug(e.getTypeName());
    System.debug(e.getCause());
}
```

Best Practice: Catch the most specific exception possible — generic `catch (Exception e)` can hide bugs.

DML Exception Pattern

```
try {
    insert accs;
} catch (DmlException e) {
    for (Integer i = 0; i < e.getNumDml(); i++) {
        System.debug('Record: ' + e.getDmlId(i));
        System.debug('Field : ' + e.getDmlFieldNames(i));
        System.debug('Msg   : ' + e.getDmlMessage(i));
    }
}
```

11. Test Classes

Salesforce **requires 75% Apex code coverage** before deploying to production. Tests ensure your code keeps working after future changes.

Anatomy of a Test Class

```
@isTest
private class AccountTriggerTest {

    @TestSetup
    static void setupData() {
        List<Account> accs = new List<Account>();
        for (Integer i = 0; i < 10; i++) {
            accs.add(new Account(Name = 'Test ' + i));
        }
        insert accs;
    }

    @isTest
    static void shouldDefaultIndustryWhenNull() {
        // Arrange
        Account a = new Account(Name='No Industry');

        // Act
        Test.startTest();
        insert a;
        Test.stopTest();

        // Assert
        Account result = [SELECT Industry FROM Account WHERE Id=:a.Id];
        System.assertEquals('Other', result.Industry, 'Default not applied');
    }
}
```

Key Annotations

- `@isTest` — marks class or method as test (not counted toward org code limit).
- `@TestSetup` — runs once before each test method, data persisted.
- `@isTest(SeeAllData=true)` — gives test access to real data (avoid; not deterministic).
- `@isTest(IsParallel=true)` — allows parallel execution.

Test.startTest / Test.stopTest

Reset governor limits between setup and the action under test. Async jobs queued inside complete synchronously when `stopTest()` is called — essential for testing batches and queueables.

Assertion Methods

```
System.assertEquals(expected, actual, 'optional message');
System.assertNotEquals(unexpected, actual);
System.assert(condition, 'message');
Assert.areEqual(expected, actual, 'msg');   // newer Assert class
```

Mocking Callouts

Real HTTP callouts aren't allowed in tests. Use `HttpCalloutMock`:

```
@isTest
global class WeatherMock implements HttpCalloutMock {
    global HTTPResponse respond(HTTPRequest req) {
        HttpResponse res = new HttpResponse();
        res.setStatusCode(200);
        res.setBody('{"temp": 25}');
        return res;
    }
}

@isTest static void testCallout() {
    Test.setMock(HttpCalloutMock.class, new WeatherMock());
    Test.startTest();
    Decimal t = WeatherService.getTemperature('London');
    Test.stopTest();
    System.assertEquals(25, t);
}
```

Best Practices

- Test positive AND negative paths (errors, exceptions, edge cases).
- Test bulk: insert 200+ records to catch bulk bugs.
- Use `@TestSetup` for shared data — runs once, faster.
- Never use `SeeAllData=true` unless you absolutely must.
- Aim for > 90% coverage in real projects (75% is the minimum, not the target).
- Always include meaningful assertions — coverage without assertions is fake testing.

12. Synchronous vs Asynchronous Operations

Synchronous

Code runs immediately, end-to-end, in the same transaction. The user (or calling system) waits for the result.

> ***Analogy:*** *Like ordering coffee at the counter and standing there until it's handed to you.*

Asynchronous

Code is queued and executed later in a background process. The user is freed instantly.

> ***Analogy:*** *You drop your shirts at the laundry, get a token, and continue with your day. The cleaning happens in the background.*

Why Go Async?

- **Higher governor limits** (e.g. SOQL queries: 100 sync vs 200 async).
- **Long-running jobs** (batch over millions of rows, scheduled nightly).
- **Mixed DML** (e.g. updating User and Account in same transaction requires async).
- **Callouts after DML** — you can't make HTTP callouts after you've done DML in a synchronous transaction; switch to `@future(callout=true)`.

Async Flavors

Type	Use Case
`@future`	Simple fire-and-forget background task
`Queueable`	Like @future but supports objects, chaining, monitoring
`Batch Apex`	Process millions of records in chunks
`Schedulable`	Run code on a schedule (cron expression)
`Platform Events`	Pub/sub event-driven architecture

Comparison Snapshot

Feature	Sync	Async
SOQL Queries	100	200
DML rows	10,000	10,000
CPU time (ms)	10,000	60,000
Heap (MB)	6	12
Callouts	100	100

13. Apex Annotations & Interfaces (@future, Batchable, Schedulable)

@future — Simplest Async

```
public class FutureDemo {
    @future(callout=true)
    public static void notifyExternalSystem(Set<Id> accIds) {
        for (Id i : accIds) {
            HttpRequest req = new HttpRequest();
            req.setEndpoint('https://api.example.com/notify/' + i);
            req.setMethod('POST');
            new Http().send(req);
        }
    }
}
// Call from anywhere:
FutureDemo.notifyExternalSystem(new Set<Id>{accId1, accId2});
```

@future Rules

- Method must be `static void`.
- Parameters must be primitives, collections of primitives, or sObjects/Ids — NOT custom objects.
- Cannot call another `@future` from a `@future` (no chaining).
- Add `(callout=true)` if you make HTTP callouts.

Batchable — Big Data Processing

Implements `Database.Batchable<sObject>`. Splits records into chunks (default 200), processes each chunk in its own transaction.

```
public class CleanInactiveAccounts implements Database.Batchable<sObject> {

    public Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator(
            'SELECT Id FROM Account WHERE LastActivityDate < LAST_N_DAYS:365'
        );
    }

    public void execute(Database.BatchableContext bc, List<Account> scope) {
        for (Account a : scope) a.Status__c = 'Inactive';
        update scope;
    }

    public void finish(Database.BatchableContext bc) {
        Messaging.SingleEmailMessage m = new Messaging.SingleEmailMessage();
        m.setToAddresses(new List<String>{'admin@x.com'});
        m.setSubject('Batch finished');
        m.setPlainTextBody('Done.');
        Messaging.sendEmail(new List<Messaging.Email>{m});
    }
}
// Run it:
Database.executeBatch(new CleanInactiveAccounts(), 200);
```

Optional Interfaces

- `Database.Stateful` — preserves instance variables across batches.
- `Database.AllowsCallouts` — enables HTTP callouts in execute.

Schedulable — Cron Jobs

```
public class WeeklyReport implements Schedulable {
    public void execute(SchedulableContext sc) {
        Database.executeBatch(new CleanInactiveAccounts());
    }
}
// Schedule (Setup or anonymous Apex):
String cron = '0 0 6 ? * MON';   // every Monday 6 AM
System.schedule('Weekly Cleanup', cron, new WeeklyReport());
```

Cron Format

`Sec Min Hour DayOfMonth Month DayOfWeek Year` (Year optional)

14. Queueable & @InvocableMethod

Queueable — The Modern @future

Implements `System.Queueable`. Advantages over @future:

- Accepts objects (not just primitives).
- Returns a Job ID you can monitor in `AsyncApexJob`.
- **Chainable** — one Queueable can enqueue the next.

```
public class ProcessAccounts implements Queueable {
    private List<Account> accs;
    public ProcessAccounts(List<Account> accs) { this.accs = accs; }

    public void execute(QueueableContext ctx) {
        for (Account a : accs) a.Description = 'Processed';
        update accs;
        // Chain another job
        if (!Test.isRunningTest()) {
            System.enqueueJob(new SendNotificationsJob(accs));
        }
    }
}
// Enqueue:
Id jobId = System.enqueueJob(new ProcessAccounts(accList));
```

> **Chaining limit:** 1 child job per parent in synchronous context; up to 5 in tests; 50 sec/250 max in production with stateful flow.

@InvocableMethod — Expose Apex to Flow

Makes an Apex method callable from Flow, Process Builder, REST. Powerful when admins want to run your logic without writing code.

```
public class GreetingService {

    @InvocableMethod(label='Generate Greeting' description='Returns hello with name')
    public static List<Output> generate(List<Input> inputs) {
        List<Output> outs = new List<Output>();
        for (Input i : inputs) {
            Output o = new Output();
            o.greeting = 'Hello, ' + i.name + '!';
            outs.add(o);
        }
        return outs;
    }

    public class Input  { @InvocableVariable public String name; }
    public class Output { @InvocableVariable public String greeting; }
}
```

Rules

- Method must be `static`, `public` or `global`.
- Only ONE `@InvocableMethod` per class.
- Input/Output must be lists (Salesforce always passes/returns bulk).

15. Deployment using Change Sets

A **Change Set** is a built-in Salesforce mechanism to move metadata from one connected org to another (e.g., Sandbox → Production).

Step-by-Step

1. **Create Deployment Connection**: Setup → Deployment Settings → enable Allow Inbound from Sandbox → Production.
2. **Create Outbound Change Set** (in source org): Setup → Outbound Change Sets → New.
3. **Add Components**: classes, triggers, fields, layouts, profiles, etc. Use "View/Add Dependencies" to catch missing pieces.
4. **Upload** to the destination org. It appears as an Inbound Change Set there.
5. **Validate** in destination (run all tests but don't commit) — recommended before peak hours.
6. **Deploy**. Production deploys require ≥ 75% Apex coverage.

Pros

- UI-driven, no setup.
- Good for occasional moves.

Cons

- Slow on big payloads.
- Cannot delete components — only add/modify.
- Cannot move profiles/permission sets cleanly.
- Only works between connected sandboxes / production.

> **Modern alternative:** SFDX + Git + CI/CD (Copado, Gearset, Bitbucket Pipelines). Treat metadata as code — deploys are repeatable, peer-reviewed, and rollback-able. Change Sets are great for learning but rarely used at scale.

16. Integration — SOAP, REST, Patterns, Postman, LLM

Integration Patterns

Pattern	Description	Example
Request & Reply	Synchronous call, wait for response	Look up shipping cost during order
Fire & Forget	Send and don't wait	Outbound platform event
Batch Data Sync	Bulk transfer on schedule	Nightly ERP sync via Bulk API
Remote Call-In	External system writes to SF	Web app creates Lead via REST
Pub / Sub	Loose event-based	Platform Events / CDC

Inbound vs Outbound

- **Inbound** — external system calls Salesforce (REST/SOAP API, Apex REST, Streaming).
- **Outbound** — Salesforce calls external system (HTTP callouts, Outbound Messages, Named Credentials).

SOAP vs REST

Aspect	SOAP	REST
Protocol	XML over HTTP/HTTPS, defined by WSDL	HTTP verbs (GET/POST/PUT/DELETE), JSON
Payload	XML (verbose)	JSON (lightweight)
Best for	Strict contracts, legacy systems, banking	Modern web/mobile apps

Outbound REST Callout

```
public class WeatherService {
    public static Decimal getTemperature(String city) {
        HttpRequest req = new HttpRequest();
        req.setEndpoint('callout:Weather_API/data?city=' + EncodingUtil.urlEncode(city, 'UTF-8'));
        req.setMethod('GET');
        req.setHeader('Content-Type', 'application/json');

        HttpResponse res = new Http().send(req);
        if (res.getStatusCode() == 200) {
            Map<String, Object> m = (Map<String,Object>)JSON.deserializeUntyped(res.getBody());
            return (Decimal) m.get('temp');
        }
        throw new CalloutException('HTTP ' + res.getStatusCode());
    }
}
```

Named Credentials (`callout:Weather_API`) hide the endpoint URL and credentials securely. Set up under Setup → Named Credentials.

Inbound — Apex REST

```
@RestResource(urlMapping='/Account/*')
global with sharing class AccountRestService {

    @HttpGet
    global static Account fetch() {
        String id = RestContext.request.requestURI.substringAfterLast('/');
        return [SELECT Id, Name, Industry FROM Account WHERE Id = :id];
    }

    @HttpPost
    global static String create(String name, String industry) {
        Account a = new Account(Name=name, Industry=industry);
        insert a;
        return a.Id;
    }
}
// Endpoint: https://yourInstance.my.salesforce.com/services/apexrest/Account/<id>
```

Postman

The standard tool for testing REST APIs.

1. Get an OAuth access token via Connected App.
2. Add header `Authorization: Bearer <token>`.
3. Hit `/services/data/v60.0/sobjects/Account` with GET/POST/etc.

Connecting to an LLM (e.g., OpenAI / Anthropic)

```
public class LLMService {
    public static String chat(String prompt) {
        HttpRequest req = new HttpRequest();
        req.setEndpoint('callout:OpenAI/v1/chat/completions');
        req.setMethod('POST');
        req.setHeader('Content-Type', 'application/json');

        Map<String, Object> body = new Map<String, Object>{
            'model'    => 'gpt-4o-mini',
            'messages' => new List<Object>{
                new Map<String, String>{'role'=>'user', 'content'=>prompt}
            }
        };
        req.setBody(JSON.serialize(body));

        HttpResponse res = new Http().send(req);
        Map<String,Object> r = (Map<String,Object>)JSON.deserializeUntyped(res.getBody());
        List<Object> choices = (List<Object>) r.get('choices');
        Map<String,Object> first = (Map<String,Object>) choices[0];
        Map<String,Object> msg = (Map<String,Object>) first.get('message');
        return (String) msg.get('content');
    }
}
```

Setup needed: Add the LLM endpoint to Remote Site Settings or (better) create a Named Credential with the API key in External Credential principal.

17. ECMA 6+ — Modern JavaScript for Salesforce Devs

Lightning Web Components are written in modern JavaScript (ES6+). You must be fluent in these features.

let vs const vs var

```
var x = 1;        // function-scoped, hoisted, can re-declare — AVOID
let y = 2;        // block-scoped, can re-assign
const z = 3;      // block-scoped, cannot re-assign

if (true) {
  let a = 10;
  var b = 20;
}
// console.log(a) → ReferenceError; console.log(b) → 20
```

> **Rule of thumb:** Use `const` by default, `let` when you need to reassign, never `var`.

Arrow Functions

```
// Traditional
function add(a, b) { return a + b; }

// Arrow
const add = (a, b) => a + b;

// Single param, no parens needed
const dbl = n => n * 2;

// Multi-line body uses { } and explicit return
const greet = name => {
  const msg = 'Hello, ' + name;
  return msg.toUpperCase();
};
```

> **Arrow vs Regular:** Arrow functions don't have their own `this` — they inherit from the surrounding scope. Critical inside class methods + callbacks.

Template Literals

```
const name = 'World';
const greeting = `Hello, ${name}!
Today is ${new Date().toDateString()}`;
```

Object Shorthand & Computed Keys

```
const name = 'Anna', age = 30;
const user = { name, age };           // shorthand

const key = 'role';
const u2 = { [key]: 'admin' };        // computed key → { role: 'admin' }
```

Spread & Rest Operators

```
// Spread — expand
const a = [1,2], b = [3,4];
const c = [...a, ...b];              // [1,2,3,4]
const obj = { ...user, role: 'admin' }; // merge objects

// Rest — collect
const sum = (...nums) => nums.reduce((s,n)=>s+n, 0);
sum(1,2,3,4);                        // 10
```

Destructuring

```
const u = { name:'Anna', age:30, city:'NYC' };
const { name, age } = u;             // Anna, 30
const { city: c = 'Unknown' } = u;   // rename + default

const [first, second, ...others] = [10,20,30,40];
```

ES6 Classes

```
class Person {
  constructor(name) { this.name = name; }
  greet() { return `Hi, I'm ${this.name}`; }
}

class Employee extends Person {
  constructor(name, role) {
    super(name);
    this.role = role;
  }
  greet() { return super.greet() + ` (${this.role})`; }
}
new Employee('Anna','Dev').greet();   // Hi, I'm Anna (Dev)
```

Default & Callback Functions

```
function greet(name='Guest') { return `Hi ${name}`; }

// Callback = function passed as argument
[1,2,3].map(n => n * 2);              // [2,4,6]
[1,2,3,4].filter(n => n % 2 === 0);   // [2,4]
```

18. Promises, Async / Await, Modules, Destructuring & JSON

Promises

A Promise represents a future value — pending → fulfilled or rejected.

```
const fetchUser = id => new Promise((resolve, reject) => {
  setTimeout(() => {
    if (id > 0) resolve({ id, name: 'Alice' });
    else reject(new Error('Bad id'));
  }, 1000);
});

fetchUser(1)
  .then(user => console.log(user))
  .catch(err  => console.error(err))
  .finally(() => console.log('done'));
```

Promise Combinators

- `Promise.all([p1,p2])` — wait for all; rejects on first failure.
- `Promise.race(...)` — first to settle wins.
- `Promise.allSettled(...)` — wait for all, never rejects.

Async / Await

Syntactic sugar over Promises. Read top-to-bottom like regular code, no chains.

```
async function showUser(id) {
  try {
    const user = await fetchUser(id);
    const orders = await fetchOrders(user.id);
    console.log(user, orders);
  } catch (e) {
    console.error('Failed:', e);
  }
}
```

> **LWC tie-in:** Imperative Apex calls return Promises, so you'll write `const result = await getAccounts();` all the time.

Modules — import / export

```
// math.js
export const PI = 3.14159;
export function square(n) { return n*n; }
export default class Calculator { /* ... */ }

// app.js
import Calculator, { PI, square } from './math.js';
```

Array Methods (Functional Style)

```
const nums = [1,2,3,4,5];
nums.map(n => n*2);                          // [2,4,6,8,10]
nums.filter(n => n > 2);                     // [3,4,5]
nums.reduce((sum,n) => sum+n, 0);            // 15
nums.find(n => n > 3);                       // 4
nums.some(n => n > 4);                       // true
nums.every(n => n > 0);                      // true
nums.forEach(n => console.log(n));
```

JSON

```
const obj  = { name:'Anna', age:30 };
const json = JSON.stringify(obj);            // '{"name":"Anna","age":30}'
const back = JSON.parse(json);               // { name:'Anna', age:30 }

// Deep copy of simple objects
const copy = JSON.parse(JSON.stringify(obj));
```

19. VS Code & SFDX Setup

What is SFDX?

Salesforce DX = a CLI + project structure that lets you treat Salesforce as a modern source-driven, Git-friendly platform.

Tools

- **Salesforce CLI** (`sf` / older `sfdx`) — command-line interface.
- **VS Code** + **Salesforce Extension Pack** — IDE.
- **Scratch Orgs / Sandboxes** — disposable / persistent dev environments.

Installation Quickstart

1. Install Node.js (v18 or later).
2. Install Salesforce CLI: `npm install -g @salesforce/cli`.
3. Install VS Code, then the "Salesforce Extension Pack" extension.

Common Commands

```
# Authorize (login) an org
sf org login web --alias DevOrg

# Create a new project
sf project generate --name MyProject

# Retrieve metadata
sf project retrieve start --metadata ApexClass:MyClass

# Deploy metadata
sf project deploy start --source-dir force-app

# Run all tests
sf apex run test --result-format human

# Execute anonymous Apex
sf apex run --file scripts/apex/hello.apex

# Open the org in browser
sf org open --target-org DevOrg
```

Project Structure

```
force-app/main/default/
├── classes/           Apex classes
├── triggers/          Apex triggers
├── lwc/               Lightning Web Components
├── aura/              Aura components
├── objects/           Custom objects + fields
├── layouts/
└── permissionsets/
```

Useful VS Code Shortcuts

- `Ctrl/Cmd + Shift + P` — open command palette → all SFDX: ... commands.
- Right-click an Apex class → "Deploy Source to Org".
- Right-click anywhere → "SFDX: Execute Anonymous Apex with Currently Selected Text".

Online Editor — Lightning Studio (or similar)

Salesforce now offers in-browser editors (Lightning Studio, Code Builder) so you can develop with no laptop install. Useful for quick fixes, but VS Code remains the production-grade IDE.

20. Introduction to LWC — Framework, Aura vs LWC, Bundle

What is LWC?

Lightning Web Components is Salesforce's modern UI framework, built on the open **Web Components** standard (Custom Elements, Shadow DOM, ES Modules). Released in 2019 as the successor to Aura.

Why LWC?

- **Performance** — runs natively in the browser, less framework overhead than Aura.
- **Standard skills** — modern JS, HTML, CSS; portable knowledge.
- **Better dev experience** — clearer separation of concerns, ES module imports.
- **Future of Salesforce UI** — all new investment is in LWC.

Aura vs LWC

Aspect	Aura	LWC
Standard	Proprietary Salesforce	Web standards
Performance	Slower (heavy framework)	Fast (browser-native)
Syntax	XML markup, Apex-like JS controllers	HTML + standard ES6+ JS
Two-way binding	Yes	One-way (explicit events to update parent)
Decorators	—	@api, @track, @wire
File extension	.cmp	.html / .js / .css

> Aura and LWC **can coexist** on the same page — you can drop an LWC inside an Aura component while you migrate.

Web Stack Behind LWC

- **Custom Elements** — define new HTML tags.
- **Shadow DOM** — encapsulates a component's DOM & CSS.
- **ES Modules** — modern import/export.
- **Templates** — re-usable HTML markup.

The LWC Bundle (Folder Structure)

An LWC is a *bundle* — one folder containing several files that share the same name as the folder.

```
helloWorld/
├── helloWorld.html       (template — required)
├── helloWorld.js         (JavaScript controller — required)
├── helloWorld.js-meta.xml (config — required)
├── helloWorld.css        (styles — optional)
└── helloWorld.svg        (icon for App Builder — optional)
```

Naming Rules

- Folder name must start with lowercase, use camelCase: `helloWorld`, `accountList`.
- Tag becomes kebab-case with namespace: `<c-hello-world>`, `<c-account-list>`.

Hello World Example

helloWorld.html

```
<template>
  <div class="slds-p-around_medium">
    <h1>Hello, {greeting}!</h1>
    <lightning-input label="Your name" value={greeting} onchange={handleChange}>
    </lightning-input>
  </div>
</template>
```

helloWorld.js

```
import { LightningElement } from 'lwc';

export default class HelloWorld extends LightningElement {
    greeting = 'World';

    handleChange(event) {
        this.greeting = event.target.value;
    }
}
```

helloWorld.js-meta.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>60.0</apiVersion>
    <isExposed>true</isExposed>
    <targets>
        <target>lightning__AppPage</target>
        <target>lightning__RecordPage</target>
        <target>lightning__HomePage</target>
    </targets>
</LightningComponentBundle>
```

SFDX Commands

```
sf lightning generate component --name helloWorld --type lwc -d force-app/main/default/lwc
sf project deploy start --source-dir force-app/main/default/lwc/helloWorld
```

21. Data Binding, Decorators, CSS, Rendering, Loops & Iteration

Data Binding

LWC binds JS class properties into the HTML template using `{property}` — **no quotes, no dollar sign**.

```
// JS
export default class Demo extends LightningElement {
    name = 'World';
    user = { firstName: 'Alice' };
}

// HTML
<p>Hello {name}</p>
<p>First name: {user.firstName}</p>
```

> No expressions inside braces! `{name + '!'}` is invalid. Compute in JS using a getter, then bind the getter.

Getter Pattern

```
get fullName() { return this.firstName + ' ' + this.lastName; }
<p>{fullName}</p>
```

Decorators

Decorator	Purpose
`@api`	Expose a property/method as part of the public API (parent can pass in).
`@track`	(rarely needed in modern LWC) Force tracking of object/array internal mutations.
`@wire`	Reactively call an Apex method or LDS adapter, auto-refresh on data change.

@api Example

```
import { LightningElement, api } from 'lwc';
export default class Greeting extends LightningElement {
    @api personName;
}
// Parent template:
<c-greeting person-name="Alice"></c-greeting>
```

@wire Example

```
import { LightningElement, wire } from 'lwc';
import getAccounts from '@salesforce/apex/AccountController.getAccounts';

export default class AccList extends LightningElement {
    @wire(getAccounts) accounts;   // auto-fires
}

<template>
  <template if:true={accounts.data}>
    <template for:each={accounts.data} for:item="a">
      <p key={a.Id}>{a.Name}</p>
    </template>
  </template>
</template>
```

Conditional Rendering

Modern syntax (API 59+): `lwc:if`, `lwc:elseif`, `lwc:else`.

```
<template lwc:if={isLoading}> Loading... </template>
<template lwc:elseif={hasError}> Error! </template>
<template lwc:else> Done! </template>
```

Older syntax (still works): `if:true`, `if:false`.

Loops with for:each

```
contacts = [
  { Id:'1', Name:'Anna' },
  { Id:'2', Name:'Bob' }
];

<template for:each={contacts} for:item="c" for:index="i">
  <p key={c.Id}>{i}: {c.Name}</p>
</template>
```

> **The `key` attribute is mandatory** on the looped element. It must be unique & stable.

Nested Loops with iterator:

```
<template iterator:it={items}>
  <li key={it.value.id}>
    <span lwc:if={it.first}>[first]</span>
    {it.value.name}
    <span lwc:if={it.last}>[last]</span>
  </li>
</template>
```

CSS Styling

Each component has its own scoped CSS thanks to Shadow DOM — your `.btn` won't bleed into other components.

```
/* myComp.css */
.title { color: #1e40af; font-weight: bold; }
:host { display: block; padding: 1rem; }   /* style the component itself */
```

SLDS — Salesforce Lightning Design System

A pre-built CSS library that ships with Salesforce. Use its utility classes for consistent look:

```
<div class="slds-card slds-p-around_medium">
  <h2 class="slds-text-heading_medium">Card title</h2>
</div>
```

22. JS Properties, Component Lifecycle & Composition

Property Types

- **Public** — decorated with `@api`, settable by parent.
- **Private** — plain class field, internal only.
- **Reactive** — any plain field is reactive automatically; the template re-renders when assigned. For *internal* mutations of objects/arrays, use `@track` or replace the whole reference.

```
export default class Sample extends LightningElement {
    @api title;          // public — parent controls
    counter = 0;         // private + reactive
    @track config = {};  // reactive on inner mutations

    increment() {
        this.counter++;  // triggers re-render
    }
}
```

Lifecycle Hooks

LWC fires hooks at specific moments in a component's life. Override them to run logic at the right time.

Hook	Fires when	Use it for
`constructor()`	Component first created	Initialize variables. NO DOM access here.
`connectedCallback()`	Component inserted into DOM	Fetch initial data, subscribe to events
`renderedCallback()`	After every render	Touch the DOM, integrate 3rd-party libs (use a flag to avoid loops!)
`disconnectedCallback()`	Removed from DOM	Clean up listeners, timers, subscriptions
`errorCallback(error, stack)`	Child component throws	Catch errors from descendants

```
export default class Demo extends LightningElement {
    hasInit = false;

    constructor() {
        super();
        console.log('1. constructor');
    }
    connectedCallback() {
        console.log('2. connected');
        this.loadData();
    }
    renderedCallback() {
        if (this.hasInit) return;
        this.hasInit = true;
        console.log('3. first render only');
    }
    disconnectedCallback() { console.log('removed'); }
    errorCallback(err, stack) { console.error('child error', err); }
}
```

> **Don't infinite-loop in renderedCallback!** Mutating reactive state inside it triggers another render, then another... Use a guard flag.

Composition

Build complex UIs by nesting components inside one another. The outer component is the *parent*, the inner is the *child*.

Example — Parent uses a Greeting Child

greeting.js (child)

```
import { LightningElement, api } from 'lwc';
export default class Greeting extends LightningElement {
    @api name;
}
```

greeting.html

```
<template><p>Hello, {name}!</p></template>
```

app.html (parent)

```
<template>
  <c-greeting name="Alice"></c-greeting>
  <c-greeting name="Bob"></c-greeting>
</template>
```

Slots — Letting Parent Project Content

```
// child card.html
<template>
  <div class="card">
    <header><slot name="header">Default Header</slot></header>
    <main><slot>Default body</slot></main>
  </div>
</template>

// parent
<c-card>
  <h1 slot="header">My Card</h1>
  <p>Body content here</p>
</c-card>
```

23. Events — Component Communication (Parent-Child, Child-Parent, LMS)

Three Communication Patterns

1. **Parent → Child**: pass via `@api` properties or call `@api` methods.
2. **Child → Parent**: dispatch a `CustomEvent` the parent listens to.
3. **Across unrelated components**: use Lightning Message Service (LMS) or pub/sub.

Parent → Child via @api Property

```
// child.js
import { LightningElement, api } from 'lwc';
export default class Child extends LightningElement {
    @api message;
}

// parent.html
<c-child message={parentMessage}></c-child>
```

Parent → Child via @api Method

```
// child.js
@api
refresh() { this.value = 0; this.fetchData(); }

// parent.js
this.template.querySelector('c-child').refresh();
```

Child → Parent via CustomEvent

```
// child.js — emit
handleClick() {
    const evt = new CustomEvent('itemselect', {
        detail: { id: this.itemId, name: this.itemName }
    });
    this.dispatchEvent(evt);
}

// parent.html — listen (lowercase, no camelCase!)
<c-child onitemselect={handleSelect}></c-child>

// parent.js
handleSelect(event) {
    console.log(event.detail.id, event.detail.name);
}
```

Bubbling & Composed

```
const evt = new CustomEvent('itemselect', {
    detail: payload,
    bubbles: true,     // climbs the DOM
    composed: true     // crosses Shadow-DOM boundaries
});
```

> Custom event names must be all lowercase, no spaces, no special chars.

Sibling/Unrelated Components — Lightning Message Service (LMS)

LMS lets components anywhere on a Lightning page (even across Aura/LWC/Visualforce) talk via a shared "channel".

Step 1 — Create a Message Channel (XML metadata)

```
// force-app/main/default/messageChannels/SampleMC.messageChannel-meta.xml
<?xml version="1.0" encoding="UTF-8"?>
<LightningMessageChannel xmlns="http://soap.sforce.com/2006/04/metadata">
    <masterLabel>SampleMC</masterLabel>
    <isExposed>true</isExposed>
    <lightningMessageFields>
        <fieldName>recordId</fieldName>
        <description>Selected record id</description>
    </lightningMessageFields>
</LightningMessageChannel>
```

Step 2 — Publisher

```
import { LightningElement, wire } from 'lwc';
import { publish, MessageContext } from 'lightning/messageService';
import SAMPLE_MC from '@salesforce/messageChannel/SampleMC__c';

export default class Publisher extends LightningElement {
    @wire(MessageContext) messageContext;

    handleClick() {
        publish(this.messageContext, SAMPLE_MC, { recordId: '001xxx' });
    }
}
```

Step 3 — Subscriber

```
import { LightningElement, wire } from 'lwc';
import { subscribe, MessageContext } from 'lightning/messageService';
import SAMPLE_MC from '@salesforce/messageChannel/SampleMC__c';

export default class Subscriber extends LightningElement {
    received;
    @wire(MessageContext) messageContext;

    connectedCallback() {
        this.subscription = subscribe(
            this.messageContext, SAMPLE_MC,
            (msg) => this.received = msg.recordId
        );
    }
}
```

Pub/Sub Module (legacy, same-page only)

Before LMS, you'd use a custom `pubsub` service. Today, LMS is preferred for everything.

24. Accessing Salesforce Data & Navigation Service

LWC offers **four** ways to read/write Salesforce data, plus a Navigation Service for moving around the UI.

1. Lightning Data Service (LDS) — UI APIs

Best for record-level UI work — Salesforce manages caching, sharing, FLS, optimistic concurrency. Zero Apex required.

Base Components — easiest of all

```
<lightning-record-form
    record-id={recordId}
    object-api-name="Account"
    layout-type="Full">
</lightning-record-form>

<lightning-record-view-form record-id={recordId} object-api-name="Account">
    <lightning-output-field field-name="Name"></lightning-output-field>
</lightning-record-view-form>

<lightning-record-edit-form record-id={recordId} object-api-name="Account">
    <lightning-input-field field-name="Name"></lightning-input-field>
    <lightning-button type="submit" label="Save"></lightning-button>
</lightning-record-edit-form>
```

Wire Adapters — finer control

```
import { LightningElement, wire } from 'lwc';
import { getRecord } from 'lightning/uiRecordApi';
import NAME_FIELD     from '@salesforce/schema/Account.Name';
import INDUSTRY_FIELD from '@salesforce/schema/Account.Industry';

export default class AccountInfo extends LightningElement {
    @api recordId;

    @wire(getRecord, { recordId: '$recordId', fields: [NAME_FIELD, INDUSTRY_FIELD] })
    account;

    get name()     { return this.account.data?.fields.Name.value; }
    get industry() { return this.account.data?.fields.Industry.value; }
}
```

2. JavaScript API — createRecord, updateRecord, deleteRecord

```
import { createRecord, updateRecord, deleteRecord } from 'lightning/uiRecordApi';
import { ShowToastEvent } from 'lightning/platformShowToastEvent';

createNew() {
    const fields = { Name: 'New Account', Industry: 'Tech' };
    createRecord({ apiName: 'Account', fields })
        .then(rec => {
            this.dispatchEvent(new ShowToastEvent({
                title: 'Created', message: 'Id ' + rec.id, variant: 'success'
            }));
        })
        .catch(err => console.error(err));
}
```

3. @wire to Apex (Reactive)

```
// AccountController.cls
public with sharing class AccountController {
    @AuraEnabled(cacheable=true)
    public static List<Account> getTopAccounts(Integer max) {
        return [SELECT Id, Name FROM Account ORDER BY AnnualRevenue DESC LIMIT :max];
    }
}

// LWC
import getTopAccounts from '@salesforce/apex/AccountController.getTopAccounts';
@wire(getTopAccounts, { max: 5 }) accounts;
```

> `cacheable=true` — required for `@wire`, must be a read-only method (no DML).

4. Imperative Apex (call when you want, e.g. on button click)

```
import getAccounts from '@salesforce/apex/AccountController.getAccounts';

async loadAccounts() {
    try {
        this.accounts = await getAccounts({ max: 10 });
    } catch (e) {
        console.error('Failed', e);
    }
}
```

Wire vs Imperative — When to Use Which

Situation	Use
Read-only, automatically refresh on cache change	@wire
Need result before DOM exists	@wire
Need to call on button click / user action	Imperative
Method does DML	Imperative (cacheable=false)

Navigation Service

The standard, declarative way to move the user to a record, list view, URL, or new-record page — works on Desktop, Mobile, Console seamlessly.

```
import { NavigationMixin } from 'lightning/navigation';

export default class NavDemo extends NavigationMixin(LightningElement) {

    goToRecord(event) {
        this[NavigationMixin.Navigate]({
            type: 'standard__recordPage',
            attributes: {
                recordId: event.target.dataset.id,
                objectApiName: 'Account',
                actionName: 'view'      // view, edit, clone
            }
        });
    }

    newAccount() {
        this[NavigationMixin.Navigate]({
            type: 'standard__objectPage',
            attributes: { objectApiName: 'Account', actionName: 'new' }
        });
    }

    listView() {
        this[NavigationMixin.Navigate]({
            type: 'standard__objectPage',
            attributes: { objectApiName: 'Account', actionName: 'list' },
            state: { filterName: 'Recent' }
        });
    }

    externalUrl() {
        this[NavigationMixin.Navigate]({
            type: 'standard__webPage',
            attributes: { url: 'https://www.salesforce.com' }
        });
    }
}
```

Page Reference Types Cheat Sheet

Type	Use For
`standard__recordPage`	View/Edit/Clone a record
`standard__objectPage`	List view / new record
`standard__navItemPage`	Custom tab
`standard__webPage`	External URL
`standard__component`	Custom LWC as page
`standard__app`	Lightning app

Final Tips for Mastery

Daily practice plan:

- **Days 24–28** — Apex foundations: write small classes, query practice in Dev Console.
- **Days 29–30** — Build at least 3 different triggers using a handler framework.
- **Days 32–35** — Async & tests: convert one of your triggers' work into a Batch + scheduled job + 100% test coverage.
- **Days 36–37** — Build one full integration: Salesforce calls a public REST API + receive a webhook back via Apex REST.
- **Days 39–41** — JavaScript drills: rewrite your favorite Apex utility class in pure JavaScript using ES6 classes.
- **Days 42+** — LWC: start with helloWorld, then a record list with @wire, then a parent-child component with events, then an LMS demo. Build a mini-app combining all four.

Top resources beyond these notes:

- Trailhead — free, hands-on Salesforce-built modules.
- Salesforce Developer Documentation (developer.salesforce.com).
- Apex Hours / SFXD Discord / SalesforceBen blog.
- Practice with a free Developer Edition org (developer.salesforce.com/signup).

— End of Notes —
Good luck on your Salesforce journey 🚀